Saving and Exporting Figures

2026-04-17

Introduction

The final step of any figure pipeline is exporting the plot at the right dimensions and resolution. It is also the step where most figures go wrong: a plot designed and previewed at 1024 × 768 pixels on screen looks fine until it is dropped into a manuscript and reprinted at journal column width, where the fonts have shrunk to illegibility and the raster edges have blurred. `ggsave()` is `ggplot2`'s export function, and the discipline to invoke it with explicit dimensions and an explicit DPI early in the workflow is what separates publication-ready figures from ones that have to be re-made the night before submission.

Prerequisites

A working knowledge of `ggplot2`, an idea of what the final figure will be used for (print, web, slide), and basic familiarity with vector vs raster graphics.

Theory

File-format choice depends on the destination:

- **PDF** is vector and infinite-resolution; it is the standard for journal submissions and the safest default unless told otherwise.
- **SVG** is also vector and ideal when the figure will be edited in Illustrator or Inkscape before final use.
- **PNG** is raster with lossless compression; use 300 dpi or higher for print, 96 dpi for web.
- **TIFF** is raster and required by several biomedical journals; use lossless `compression = "lzw"` to avoid quality loss.
- **EPS** is vector and required by a shrinking minority of journals; PDF is generally a better choice when allowed.

Dimensions are specified through `width` and `height` with units (`"in"`, `"cm"`, `"mm"`); DPI applies only to raster formats. Font sizes scale with `base_size` in the theme; setting `base_size = 12` against a 6 × 4 inch figure produces fonts that print legibly at journal column width.

Assumptions

The graphics device for the chosen format is available (Cairo for PNG with anti-aliasing, `ragg` for high-quality raster output, `svglite` for compact SVG); modern R installations include all of these but verify if a specific format fails.

R Implementation

```
library(ggplot2)

p <- ggplot(mtcars, aes(wt, mpg)) +
  geom_point(size = 3) +
  theme_minimal(base_size = 12)

# Standard PDF for journal submission
```

```r
ggsave("figure1.pdf", p, width = 6, height = 4, units = "in")

# High-resolution PNG
ggsave("figure1.png", p, width = 6, height = 4, units = "in", dpi = 300)

# Journal-standard column width (typically 85 mm for single column)
ggsave("figure1.tiff", p, width = 85, height = 60, units = "mm",
       dpi = 300, compression = "lzw")

# Vector SVG for editing
ggsave("figure1.svg", p, width = 6, height = 4)
```

Output & Results

Four export variants of the same `ggplot` object: a PDF for journal submission, a 300-dpi PNG for web embedding, an LZW-compressed TIFF at 85 mm single-column width, and an SVG for downstream editing. Each file is identical in content but differs in format-specific properties.

Interpretation

A reporting sentence (methods or figure-preparation note): "All figures were rendered with `ggplot2` 3.5 (`base_size = 12`) and exported to PDF (vector) and TIFF (LZW-compressed, 300 dpi) at single-column width (85 mm); raster previews use the `ragg` device for anti-aliased text." Documenting the export pipeline aids reproducibility and pre-empts reviewer questions about figure resolution.

Practical Tips

- Always export at the final publication size; resizing a saved figure in a word processor distorts fonts and produces blurry raster edges.
- For multi-panel figures composed with `patchwork`, export the composed object directly; do not save panels separately and stitch them in an editor.
- Use `ragg::agg_png()` (or set `options(ragg.background = "white")`) for anti-aliased PNG output that handles UTF-8 and emoji correctly.
- For TIFF submissions, `compression = "lzw"` produces lossless files at typically half the size of uncompressed; some journals also accept `"zip"`.
- Save a small reproducible script next to every published figure (`figure1.R`); regenerating a figure six months later is otherwise an exercise in archaeology.
- For vector figures with thousands of overlapping elements (large hexbins, dense scatter), file sizes balloon; rasterise the busy layer with `ggrastr::rasterise()` while keeping axes and text vector for a hybrid output that prints cleanly.

R Packages Used

`ggplot2` for `ggsave()`; `ragg` for fast, high-quality raster devices; `svglite` for compact SVG; `Cairo` for legacy anti-aliased rendering; `ggrastr` for selectively rasterising vector layers in otherwise vector exports.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**

- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts